

회계가 바로 서야 경제가 바로 섭니다.

제08회 재무빅데이터분석사 자격시험

FDA 1급

Financial big-Data Analyst

- 시험시간: 180분
- 객관식배점: 문항별 배점 참조
- 주관식배점: 문항별 배점 참조

- 회계처리기준 등을 적용하여 정답을 구하여야 하는 문제는 시험시행 공고일 현재 시행 중인 법률·기준 등을 적용하여 그 정답을 구하여야 합니다.
- 본 기출문제, 해설, 답안은 저작권법에 의해 보호받으므로 무단복제와 무단전제를 금합니다.

KICPA 한국공인회계사회

이론 객관식

01 다음과 같이 2차원 Array를 만들었다.

```
import numpy as np
arr = np.array([[5, 12, 8, 3],
               [9, 4, 15, 1],
               [6, 11, 2, 10],
               [14, 7, 13, 16]])
```

상기 2차원 Array를 다음과 같이 출력하고 싶다.

```
[[3, 5, 8, 12],
 [1, 4, 9, 15],
 [2, 6, 10, 11],
 [7, 13, 14, 16]]
```

상기와 같이 출력하기 위한 코드로 옳은 것을 고르시오. (3점)

㉠	arr = np.sort(arr) print(arr)
㉡	for i in range(5): arr[i] = np.sort(arr[i]) print(arr)
㉢	arr.sort(axis=0) arr.sort(axis=1) print(arr)
㉣	arr.sort(axis=1) arr.sort(axis=0) print(arr)
㉤	arr = np.sort(arr, axis=1) arr = np.sort(arr, axis=0) print(arr)

① ㉠

② ㉡

③ ㉢

④ ㉣

⑤ ㉤

해설

㉠ 순서대로 실행하면, 목표 결과를 얻을 수 있다.

🔒 정답 :

①

02 코드의 출력 결과가 다음과 같을 때 다음 보기 중 ㉠, ㉡, ㉢, ㉣에 모두 올바른 값이 들어간 것은? (3점)

```
import numpy as np
np.random.seed(9)
arr = np.random.randint(1, 150, size=(4, 3, 4))
filtered = arr[(arr >= ㉠) & (arr <= ㉡)]
print(filtered.reshape(㉢, ㉣))
[[57 66 60 41 63 57 50]
 [59 64 41 53 75 41 73]]
```

㉠	㉠ : 41,	㉡ : 75,	㉢ : 7,	㉣ : 2
㉡	㉠ : 50,	㉡ : 70,	㉢ : 2,	㉣ : 7
㉢	㉠ : 41,	㉡ : 75,	㉢ : 2,	㉣ : 7
㉣	㉠ : 1,	㉡ : 149,	㉢ : 2,	㉣ : 7
㉤	㉠ : 41,	㉡ : 75,	㉢ : 1,	㉣ : 14

① ㉠

② ㉡

③ ㉢

④ ㉣

⑤ ㉤

해설

㉤ 순서대로 실행하면, 목표 결과를 얻을 수 있다.

 정답 : ③

03 다음은 특정 기간의 거래 내역 데이터프레임인 df이다.

```
print(df)
  ID  Type  Amount  Taxable
0   1  Sales   2000     True
1   2  Expense  -500    False
2   3  Sales   3000     True
3   4  Service  1500    False
4   5  Expense  -800    False
```

'Amount' 칼럼이 양수이고 'Taxable' 칼럼이 True인 거래에 대해서만 10%의 세금을 계산하여 'TaxAmt' 칼럼에 추가하려고 하며, 그렇지 않은 거래는 행만 'TaxAmt' 칼럼에서 0으로 넣는다. 다음은 이를 코드로 작성한 것이다.

```
import pandas as pd

df['TaxAmt'] = 0.0

for a in range(len(df)):
    if df.iloc[a, b] > 0 and df.iloc[a, c] == True:
        df.iloc[a, d] = df.iloc[a, e] * f
```

㉠ ~ ㉦에 들어갈 내용으로 옳은 것을 고르시오. (3점)

㉠	a : i, b : 2, c : 3, d : 4, e : 2, f : 0.1
㉡	a : i, b : 'Amount', c : 'Taxable', d : 4, e : 'Amount', f : 0.1
㉢	a : i, b : 2, c : 3, d : 'TaxAmt', e : 2, f : 0.1
㉣	a : i, b : 3, c : 2, d : 4, e : 3, f : 0.1
㉤	a : i, b : 2, c : 4, d : 2, e : 3, f : 10%

① ㉠

② ㉡

③ ㉢

④ ㉣

⑤ ㉤

해설

㉠ 순서대로 실행하면, 목표 결과를 얻을 수 있다.

정답 :

①

04 다음은 직원별 성과를 나타내는 데이터프레임 df이다.

```
print(df)
      직원  성과점수
0   Alice    110
1    Bob     95
2  Charlie   150
3   David     80
```

각 직원에게는 기본 보너스(base_bonus)를 지급하되, 목표(bonus_target)를 초과 달성한 경우 추가 보너스(extra_bonus)를 더하여 '총보너스' 칼럼에 저장하려고 한다. 다음은 이를 위한 코드이다.

```
import pandas as pd

bonus_target = 100
base_bonus = 500
extra_bonus = 300

def calculate_bonus(㉑):
    score = ㉒['성과점수']
    current_bonus = ㉓

    if score > ㉔:
        current_bonus = current_bonus + extra_bonus

    return ㉕

df['총보너스'] = df.apply(calculate_bonus, axis=㉖)
print(df)
      직원  성과점수  총보너스
0   Alice    110    800
1    Bob     95    500
2  Charlie   150    800
3   David     80    500
```

㉑ ~ ㉖에 들어갈 내용으로 옳은 것을 고르시오. (3점)

㉑	㉑ : emp ㉒ : extra_bonus	㉓ : bonus_target ㉔ : 1	㉕ : base_bonus ㉖ : 1
㉑	㉑ : emp ㉒ : current_bonus	㉓ : base_bonus ㉔ : 1	㉕ : extra_bonus ㉖ : 1
㉑	㉑ : emp_data ㉒ : current_bonus	㉓ : base_bonus ㉔ : 1	㉕ : bonus_target ㉖ : 1
㉑	㉑ : emp_data ㉒ : current_bonus	㉓ : bonus_target ㉔ : 1	㉕ : bonus_target ㉖ : 1
㉑	㉑ : emp_data ㉒ : bonus_target	㉓ : base_bonus ㉔ : 0	㉕ : bonus_target ㉖ : 0

① 가
④ 라

② 나
⑤ 마

③ 다

해설

㉠ 순서대로 실행하면, 목표 결과를 얻을 수 있다.

정답 : ③

05 다음은 판다스의 `pivot_table()` 함수를 사용하여 제품별/지역별 판매 데이터 분석을 수행하는 코드이다.

```
import pandas as pd
import numpy as np

data = {
    '날짜': ['2023-01-15', '2023-01-15', '2023-01-15',
            '2023-01-15', '2023-01-16', '2023-01-16',
            '2023-01-16', '2023-01-16', '2023-01-17',
            '2023-01-17', '2023-01-17', '2023-01-17'],
    '제품': ['A', 'B', 'A', 'B', 'A', 'B',
            'A', 'B', 'A', 'B', 'A', 'B'],
    '지역': ['서울', '서울', '부산', '부산', '서울', '서울',
            '부산', '부산', '서울', '서울', '부산', '부산'],
    '판매량': [100, 150, 80, 120, 110, 140,
              90, 130, 120, 160, 85, 125],
    '금액': [5000, 9000, 4000, 7200, 5500, 8400,
            4500, 7800, 6000, 9600, 4250, 7500]
}

df = pd.DataFrame(data)
df['날짜'] = pd.to_datetime(df['날짜'])
df['요일'] = df['날짜'].dt.day_name()

result = pd.pivot_table(df,
                        values=㉠,
                        index=[㉡, ㉢],
                        columns=㉣,
                        aggfunc=㉤)

print(result)
```

위 코드를 실행했을 때, 아래와 같이 제품별, 지역별로 각 집계 데이터가 출력되게 하려고 한다.

제품	지역	요일	금액		판매량	
			A	B	A	B
부산		Monday	4500	7800	90	130
		Sunday	4000	7200	80	120
		Tuesday	4250	7500	85	125
서울		Monday	5500	8400	110	140
		Sunday	5000	9000	100	150
		Tuesday	6000	9600	120	160

㉠ ~ ㉤ 에 들어갈 내용으로 옳은 것으로 묶인 것을 고르시오. (3점)

㉠	㉠ : ['판매량', '금액'] ㉡ : '제품'	㉢ : '지역' ㉣ : 'mean'	㉤ : '요일'
㉡	㉠ : ['판매량', '금액'] ㉡ : '요일'	㉢ : '제품' ㉣ : 'sum'	㉤ : '지역'
㉢	㉠ : ['판매량', '금액'] ㉡ : '제품'	㉢ : '지역' ㉣ : 'count'	㉤ : '요일'
㉣	㉠ : ['판매량', '금액'] ㉡ : '요일'	㉢ : '지역' ㉣ : np.mean	㉤ : '제품'
㉤	㉠ : ['판매량', '금액'] ㉡ : '제품'	㉢ : '요일' ㉣ : np.sum	㉤ : '지역'

① ㉠

② ㉡

③ ㉢

④ ㉣

⑤ ㉤

해설

㉠ 순서대로 실행하면, 목표 결과를 얻을 수 있다.

🔑 정답 : ①

06 다음은 2개의 데이터프레임을 생성하는 코드이다.

```
import pandas as pd

coa_data_kr = {'계정코드': [101, 102, 201, 202, 401, 402, 501, 502, 503, 601],
               '계정명': ['현금', '매출채권', '매입채무', '미지급급여', '상품매출', '용역수익', '임차료', '급여', '수도광열비', '지급수수료'],
               '계정유형': ['자산', '자산', '부채', '부채', '수익', '수익', '비용', '비용', '비용', '비용'],
               '통계계정여부': [False, True, True, False, False, False, False, True, False, False]}

df_계정원장 = pd.DataFrame(coa_data_kr)

gl_data_kr = {'전표번호': range(1001, 1011),
               '거래일자': pd.to_datetime(['2023-11-01', '2023-11-01', '2023-11-05',
                                             '2023-11-10', '2023-11-10', '2023-11-15', '2023-11-20', '2023-11-25',
                                             '2023-11-30', '2023-11-30']),
               '연결계정코드': [101, 401, 102, 201, 501, 101, 502, 999, 102, 201],
               '차변금액': [10000, 0, 5000, 0, 2000, 0, 8000, 300, 1500, 0],
               '대변금액': [0, 10000, 0, 5000, 0, 2000, 0, 0, 0, 1500],
               '적요': ['초기자본입금', '상품매출기록', '용역대금청구', '매입대금지급', '임차료계상',
                        '용역대금회수', '급여지급', '오류전표', '채권회수', '채무지급']}

df_분개장 = pd.DataFrame(gl_data_kr)
```

상기 2개의 데이터프레임을 조인한 결과가 다음과 같다.

	전표번호	거래일자	연결계정코드	차변금액	대변금액	적요	계정코드	계정명	계정유형
0	1001	2023-11-01	101	10000	0	초기자본입금	101.0	현금	자산
1	1002	2023-11-01	401	0	10000	상품매출기록	401.0	상품매출	수익
2	1003	2023-11-05	102	5000	0	용역대금청구	102.0	매출채권	자산
3	1004	2023-11-10	201	0	5000	매입대금지급	201.0	매입채무	부채
4	1005	2023-11-10	501	2000	0	임차료계상	501.0	임차료	비용
5	1006	2023-11-15	101	0	2000	용역대금회수	101.0	현금	자산
6	1007	2023-11-20	502	8000	0	급여지급	502.0	급여	비용
7	1008	2023-11-25	999	300	0	오류전표	NaN	NaN	NaN
8	1009	2023-11-30	102	1500	0	채권회수	102.0	매출채권	자산
9	1010	2023-11-30	201	0	1500	채무지급	201.0	매입채무	부채

상기 결과를 만들기 위해 코드는 다음과 같다.

```
merged_df_q1 = pd.merge(df_분개장,
                        df_계정원장[[㉠, ㉢, ㉤]],
                        how =㉡,
                        left_on = ㉣,
                        right_on = ㉦)

print(merged_df_q1)
```

㉠ ~ ㉦ 에 들어갈 내용으로 옳은 것으로 묶인 것을 고르시오. (3점)

㉠	㉠ : '계정코드'	㉢ : '계정명'	㉤ : '계정유형'
	㉡ : 'right'	㉣ : '계정코드'	㉦ : '연결계정코드'
㉢	㉠ : '계정명'	㉢ : '계정코드'	㉤ : '계정유형'
	㉡ : 'right'	㉣ : '계정코드'	㉦ : '연결계정코드'
㉤	㉠ : '계정명'	㉢ : '계정코드'	㉤ : '계정유형'
	㉡ : 'left'	㉣ : '계정코드'	㉦ : '연결계정코드'
㉥	㉠ : '계정코드'	㉢ : '계정명'	㉤ : '계정유형'
	㉡ : 'left'	㉣ : '계정코드'	㉦ : '연결계정코드'
㉦	㉠ : '계정코드'	㉢ : '계정명'	㉤ : '계정유형'
	㉡ : 'left'	㉣ : '연결계정코드'	㉦ : '계정코드'

① ㉠

② ㉢

③ ㉤

④ ㉡

⑤ ㉦

해설

㉠ 순서대로 실행하면, 목표 결과를 얻을 수 있다.

정답 :

⑤

- 07 HTML과 유사한 태그에서 내용을 추출하려고 한다. 텍스트 내 <tag>...</tag> 형식의 모든 부분을 찾되, 태그 사이에 가장 짧게 매칭되는 내용만을 추출하고자 한다. 다음을 이를 위한 코드이다. 아래 코드의 ㉠에 들어갈 *가장 적절한* 정규표현식을 고르시오.
(3점)

```
import re

text = "<tag>Content 1</tag> some text <tag>Content 2</tag>"

pattern = ㉠

results = re.findall(pattern, text)

print(results)
['Content 1', 'Content 2']
```

- ① `r"<tag>(.*?)</tag>"`
- ② `r"<tag>(.*?)</tag>"`
- ③ `r"<tag>([<])</tag>"`
- ④ `r"<tag>(.*?)</tag>"`
- ⑤ `r"<tag>(.*?)</tag>"`

해설

보기 ②를 실행하면, 목표 결과를 얻을 수 있다.

 정답 : ②

08 Fraudit 교육용 버전 왼쪽 네비게이터의 Tables 트리의 samples 폴더안에 있는 Bid.tbl을 연다. 그리고 다음과 같이 이를 데이터프레임으로 변환한다.

```
>>> df = Bid.toDF()
>>> df
```

	입찰번호	프로젝트번호	입찰자번호	입찰일자	입찰금액
0	5001	9001	8048	2009-03-07	188163.44
1	5002	9001	8063	2009-02-03	383786.19
2	5003	9001	8036	2009-02-03	593798.81
3	5004	9001	8059	2009-02-22	796503.19
4	5005	9002	8019	2009-02-11	772554.69
..	...	...	...	...	...
274	5275	9063	8058	2009-02-28	1312109.38
275	5276	9064	8006	2009-12-18	619359.56
276	5277	9064	8066	2009-12-18	1199294.32
277	5278	9064	8014	2009-12-18	1797663.21
278	5279	9064	8041	2009-12-18	2419713.88

[279 rows x 5 columns]

각 '프로젝트번호' 그룹 내에서 개별 '입찰금액'이 해당 프로젝트의 평균 입찰금액과 얼마나 차이 나는지를 계산하여 '평균대비차액'이라는 새 컬럼에 추가하려고 한다. 다음 코드의 ㉠에 들어갈 *가장 적절한* 코드는 무엇인가? (3점)

```
>>> import pandas as pd
>>> gb_project = df.groupby('프로젝트번호')['입찰금액']
>>> df['평균대비차액'] = ㉠
```

- ① gb_project['입찰금액'].transform(lambda x: x.mean() - x)
- ② gb_project.agg(lambda x: x - x.mean())
- ③ df_bid['입찰금액'] - gb_project.mean()
- ④ gb_project.transform(lambda x: x - x.mean())
- ⑤ gb_project['입찰금액'].transform(lambda x: x - x.mean())

해설

보기 ④를 실행하면, 목표 결과를 얻을 수 있다.

 정답 : ④

09 다음 코드를 실행했을 때의 설명으로 옳은 것을 고르시오. (3점)

```
import networkx as nx

G = nx.Graph()
G.add_edges_from([(1, 2, {"weight": 1}), (2, 3, {"weight": 2}), (3, 4, {"weight": 3}), (4, 1, {"weight": 4})])

G.add_edge(3, 5, weight=5)

path_1_4_initial = nx.has_path(G, 1, 4)

G.remove_edge(3, 4)

num_nodes_final = G.number_of_nodes()
num_edges_final = G.number_of_edges()
path_1_4_final = nx.has_path(G, 1, 4)
degree_3_final = G.degree(3)
components = list(nx.connected_components(G))
```

- ① path_1_4_initial은 False이고, path_1_4_final은 True이다.
- ② remove_edge(3, 4)는 엣지뿐만 아니라 연결된 노드 3과 4도 함께 제거하므로, num_nodes_final은 3이 된다.
- ③ 엣지 (3, 4) 제거 후, 노드 1과 4 사이에는 경로 (1-2-3-5)가 존재하므로 path_1_4_final은 False이다.
- ④ degree_3_final은 2이며, 그래프 G는 여전히 하나의 연결된 컴포넌트(connected component)를 유지한다.
- ⑤ num_edges_final은 3이며, degree_3_final은 1이다.

해설

- ① 초기 그래프에는 (1, 4) 엣지와 (1-2-3-4) 경로가 모두 존재하므로 path_1_4_initial은 True이다. 엣지 (3, 4) 제거 후에도 경로 (1-4)는 여전히 존재하므로 path_1_4_final도 True이다.
- ② remove_edge 함수는 명시된 엣지만 제거하고, 해당 엣지에 연결된 노드는 제거하지 않는다. 초기 노드는 1, 2, 3, 4, 5로 총 5개였으며, 노드 제거가 없었으므로 num_nodes_final은 5가 되어야 한다.
- ③ 엣지 (3, 4) 제거 후 노드 1과 4 사이에는 직접적인 엣지 (1, 4)가 여전히 존재하므로 path_1_4_final은 True이다. 제시된 경로 (1-2-3-5)는 노드 4로 연결되지 않는다.
- ④ 노드 3은 초기에 2, 4, 5와 연결되어 차수가 3이었다. 엣지 (3, 4)가 제거되면 노드 3은 2와 5에만 연결되므로 degree_3_final은 2가 된다. 엣지 (3, 4)를 제거해도 다른 경로들(예: 1-2-3-5, 1-4)이 존재하여 모든 노드(1, 2, 3, 4, 5)가 서로 연결된 상태를 유지하므로, 그래프는 여전히 하나의 연결된 컴포넌트를 유지한다.
- ⑤ 초기 엣지는 (1, 2), (2, 3), (3, 4), (4, 1), (3, 5)로 총 5개였다. (3, 4) 엣지를 제거하면 4개의 엣지가 남으므로 num_edges_final은 4이다. 위에서 계산했듯이 degree_3_final은 2이다.

정답 : ④

10 다음은 데이터프레임을 생성하는 코드이다. 이 코드를 실행한 결과는 아래와 같다.

```
import pandas as pd

총_감사대상_금액 = 0.0
총_감사_실시_건수 = 0

data = {
    '거래ID': [㉠, 202, 203, 204, 205],
    '계정': ['매출', '매출원가', '판매관리비', '영업외수익', '법인세'],
    '금액': [50000, 20000, ㉡, 8000, 7000],
    '감사필요': [True, False, True, False, True],
    '중요도': ['상', '하', '중', '하', '상']
}

거래_내역 = pd.DataFrame(data)

def check_and_update_audit_status(transaction_data):
    global 총_감사_실시_건수
    if transaction_data['감사필요']:
        print("거래 {} 감사 확인됨.".format(transaction_data['거래ID']))
        총_감사_실시_건수 = 총_감사_실시_건수 + 1
        return True
    return False

총_감사대상_금액 = 거래_내역.loc[거래_내역['감사필요'] == True, ㉢].sum()

감사_실시_결과 = 거래_내역.apply(check_and_update_audit_status, axis=1)

print(㉣)
print(㉤)
거래 201 감사 확인됨.
거래 203 감사 확인됨.
거래 205 감사 확인됨.
72000
3
```

㉠ ~ ㉤ 에 들어갈 내용으로 옳은 것으로 묶인 것을 고르시오. (3점)

㉠	㉠ : 201 ㉡ : 총_감사대상_금액	㉢ : 15000 ㉣ : 총_감사_실시_건수	㉤ : '금액'
㉡	㉠ : 203 ㉡ : 총_감사_실시_건수	㉢ : 72000 ㉣ : 총_감사대상_금액	㉤ : '중요도'
㉢	㉠ : 205 ㉡ : 총_감사대상_금액	㉢ : 72000 ㉣ : 총_감사_실시_건수	㉤ : '계정'
㉣	㉠ : 201 ㉡ : 총_감사_실시_건수	㉢ : 72000 ㉣ : 총_감사대상_금액	㉤ : '금액'
㉤	㉠ : 203 ㉡ : 총_감사대상_금액	㉢ : 15000 ㉣ : 총_감사_실시_건수	㉤ : '금액'

① ㉠

② ㉡

③ ㉢

④ ㉣

⑤ ㉤

해설

㉠ 순서대로 실행하면, 목표 결과를 얻을 수 있다.

🔒 정답 :

①

주관식

11~12

당신은 (주)한공의 내부감사인이다. (주)한공의 결산일은 2018년 12월 31일이다. (주)한공의 회계프로그램에서 2018년 외상매출금 관련 분개장 및 거래처별 외상매출금 잔액 데이터를 입수하여 연령분석을 하려고 한다. 2018년 외상매출금 관련 분개장과 외상매출금 잔액 데이터는 아래에서 다운로드 받을 수 있다.

[외상매출금.tbl](#)

[외상매출금잔액.tbl](#)

테이블을 불러온 후 구조를 살펴보면 다음과 같다.

[외상매출금.tbl]

칼럼명	데이터 타입	설명
전표일자	String	전표일자
전표번호	Integer	전표번호
계정코드	Integer	계정과목을 나타내는 코드
계정과목	String	계정과목
차변금액	Integer	차변금액
대변금액	Integer	대변금액
거래처	String	거래처코드

[외상매출금잔액.tbl]

외상매출금잔액.tbl은 전기이월된 거래처별 외상매출금 잔액을 나타낸다.

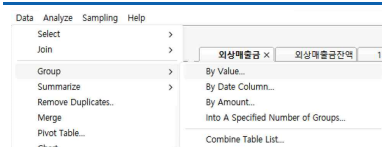
칼럼명	데이터 타입	설명
거래처	String	전표일자
전기이월	Integer	외상매출금 전기이월금액

예를 들어, 2018년 12월 31일 현재 외상매출금 잔액이 200원이라고 가정하자. 이 중 2018년 12월 1일에 외상매출금이 100원이 차변에 발생되었고 2018년 11월 1일에 외상매출금이 150원 차변에 발생하였다면, 외상매출금 잔액 200원 중 100원은 2018년 12월 1일에 발생한 것이며 나머지 100원은 2018년 11월 1일에 발생한 것이므로 연체일수는 회수하지 못한 금액이 포함된 가장 이른 2018년 11월 1일을 기준으로 계산한다(즉 외상매출금 회수는 선입선출이라고 가정한다).

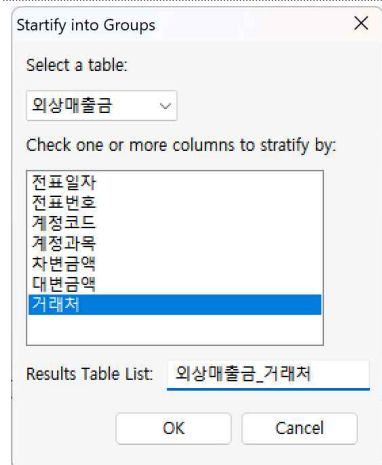
- 11 2018년 12월 31일을 결산일이라고 할 때 외상매출금 잔액이 있는 거래처의 개수를 숫자만 입력하라. 천단위 콤마는 제외한다. (4점)

정답 : 22

해설



■ Data >> Group >> By Value 메뉴를 누른다.

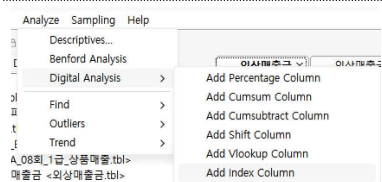


■ Select a table : 외상매출금 선택

■ Check one or more columns to stratify by : 거래처 체크

■ Results Table List : 외상매출금_거래처 입력

■ OK 버튼을 누른다.



■ Analyze >> Digital Analysis >> Add Vlookup 메뉴를 누른다.

Add Vlookup Column

Table: 외상매출금

Column: 거래처

Refer Table: 외상매출금잔액

Refer Column: 거래처

Refer Value Column: 전기이월

New Column Name: 전기이월

OK Cancel

- Table : 외상매출금 선택
- Column : 거래처 선택
- Refer Table : 외상매출금잔액 선택
- Refer Column : 거래처 선택
- Refer Value Column : 전기이월 선택
- New Column Name : 전기이월 입력
- OK 버튼을 누른다.

외상매출금 외상매출금잔액 외상매출금_거래처

Filter Expression: 0 of 35

	전표일자	전표번호	계정코드	계정과목	차변금액	대변금액	거래처	전기이월
0	2018-02-28	21	108	외상매출금	0	2,640,000	00113	2640000
1	2018-02-28	50008	108	외상매출금	2,640,000	0	00113	2640000
2	2018-05-31	55	108	외상매출금	0	2,640,000	00113	2640000
3	2018-05-31	50006	108	외상매출금	2,640,000	0	00113	2640000
4	2018-07-16	21	108	외상매출금	0	2,640,000	00113	2640000
5	2018-07-31	50007	108	외상매출금	2,640,000	0	00113	2640000
6	2018-08-31	54	108	외상매출금	0	2,640,000	00113	2640000
7	2018-08-31	50018	108	외상매출금	2,640,000	0	00113	2640000
8	2018-09-21	7	108	외상매출금	0	2,640,000	00113	2640000
9	2018-09-28	50023	108	외상매출금	2,640,000	0	00113	2640000

■ vlookup으로 갖고 온 전기이월 칼럼이 삽입된다.

Analyze Sampling Help

Descriptives... Benford Analysis Digital Analysis Find Outliers Trend

외상매출금 외상매출금

Add Percentage Column Add Cumsum Column Add Cumsubtract Column Add Shift Column Add Vlookup Column

- Analyze >> Digital Analysis >> Add Cumsum Column 메뉴를 누른다.

Add Cumsum Column

Table: 외상매출금_거래처

Column: 차변금액

New Column Name: 차변금액_누적

OK Cancel

- Table : 외상매출금_거래처 선택
- Column : 차변금액 선택
- New Column Name : 차변금액_누적 입력
- OK 버튼을 누른다.

Analyze Sampling Help

Descriptives... Benford Analysis Digital Analysis Find Outliers Trend

외상매출금 외상매출금

Add Percentage Column Add Cumsum Column Add Cumsubtract Column Add Shift Column Add Vlookup Column

- Analyze >> Digital Analysis >> Add Cumsum Column 메뉴를 누른다.

Add Cumsum Column

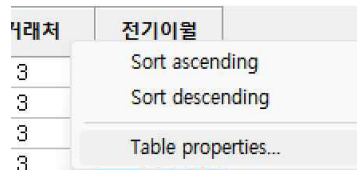
Table: 외상매출금_거래처

Column: 대변금액

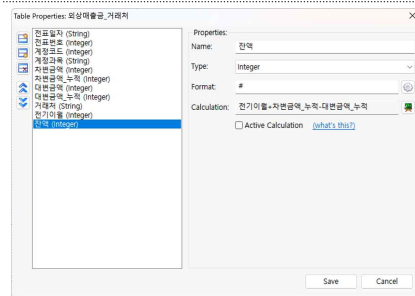
New Column Name: 대변금액_누적

OK Cancel

- Table : 외상매출금_거래처 선택
- Column : 대변금액 선택
- New Column Name : 대변금액_누적 입력
- OK 버튼을 누른다.



■ **외상매출금_거래처** 테이블에서 오른쪽 클릭하고 Table properties를 누른다.



■ **잔액** 칼럼을 삽입한다.

- Name : 잔액 입력
- Type : Integer 선택
- Format : # 선택
- Calculation : 다음과 같이 입력한다.

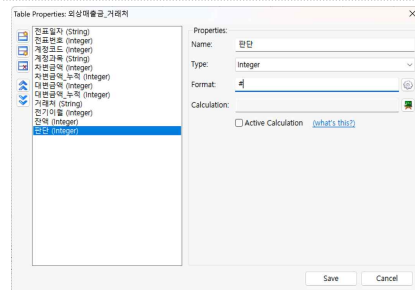
전기이월+차변금액_누적-대변금액_누적

■ Save 버튼을 누른다.

Filter Expression: Set 0 of 35

	잔표일자	잔표번호	계정코드	계정과목	자변금액	자변금액_누적	대변금액	대변금액_누적	거래처	전기이월	잔액
0	2018-02-28		21	108 외상매출금	0	0	2,640,000	2640000	00113	2640000	0
1	2018-02-28	50008		108 외상매출금	2,640,000	2640000	0	2640000	00113	2640000	2640000
2	2018-05-31	55		108 외상매출금	0	2640000	2,640,000	5280000	00113	2640000	0
3	2018-05-31	50006		108 외상매출금	2,640,000	5280000	0	5280000	00113	2640000	2640000
4	2018-07-16	21		108 외상매출금	0	5280000	2,640,000	7920000	00113	2640000	0
5	2018-07-31	50007		108 외상매출금	2,640,000	7920000	0	7920000	00113	2640000	2640000
6	2018-08-31	54		108 외상매출금	0	7920000	2,640,000	10560000	00113	2640000	0
7	2018-08-31	50018		108 외상매출금	2,640,000	10560000	0	10560000	00113	2640000	2640000
8	2018-09-21	7		108 외상매출금	0	10560000	2,640,000	13200000	00113	2640000	0
9	2018-09-28	50023		108 외상매출금	2,640,000	13200000	0	13200000	00113	2640000	2640000

■ 잔액 칼럼의 마지막 레코드가 0을 초과하는 서브 테이블을 구하면 된다.



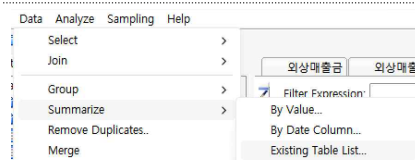
■ **판단** 칼럼을 삽입한다.

- Name : 판단 입력
- Type : Integer 선택
- Format : # 선택

■ Save 버튼을 누른다.

■ 코드 창을 열고 다음과 같이 입력하고 영역을 설정한 후 실행한다.

```
for i in range(len(외상매출금_거래처)) :
    for j in range(len(외상매출금_거래처[i])) :
        if 외상매출금_거래처[i][len(외상매출금_거래처[i])-1]['잔액'] > 0 :
            외상매출금_거래처[i][j]['판단'] = 1
        else :
            외상매출금_거래처[i][j]['판단'] = 0
```



■ Data >> Summarize >> Existing Table List 메뉴를 누른다.

Summarize List Of Tables

Table List: 외상매출금_거래처

Summarizing Expressions:

Calculate Max on 판단, or

enter an expression: 잔액판단="max(group['판단'])"

Remove

Results Name: 잔액거래처

OK Cancel

- Table List : 외상매출금_거래처
- Summarizing Expressions :
 - Calculate Max on 판단
 - Add 버튼을 누르고 잔액판단이라고 입력한다.
- Results Name : 잔액거래처 입력
- OK 버튼을 누른다.

Filter Expression: 잔액판단 > 0

	StartValue	EndValue	잔액판단
0	00113	00113	1.0
1	00114	00114	0.0
2	00115	00115	1.0
3	00116	00116	0.0
4	00117	00117	0.0
5	00118	00118	1.0
6	00119	00119	1.0
7	00120	00120	1.0

- Filter Expression에 다음과 같이 입력한 후 Set 버튼을 누른다.

잔액판단 > 0

	StartValue	EndValue	잔액판단
0	00113	00113	1.0
1	00115	00115	1.0
2	00118	00118	1.0
3	00119	00119	1.0
4	00120	00120	1.0
5	00121	00121	1.0
6	00122	00122	1.0
7	00124	00124	1.0
8	00127	00127	1.0
9	00128	00128	1.0
10	00129	00129	1.0
11	00131	00131	1.0
12	00132	00132	1.0
13	00141	00141	1.0
14	00152	00152	1.0
15	00227	00227	1.0
16	00344	00344	1.0
17	00412	00412	1.0
18	00432	00432	1.0
19	00469	00469	1.0
20	00565	00565	1.0
21	00639	00639	1.0

- 거래처 개수가 22개임을 확인할 수 있다.

12 2018년 12월 31일을 결산일이라고 할 때 연체일수가 200일이 넘는 거래처의 개수를 숫자만 입력하라. (6점)

 정답 : 2

해설

- (1) **New Script** 아이콘을 선택한다.
(2) 아래와 같이 코드를 입력하고 해당 영역을 선택한 후  를 누른다.

```
# DataFrame 전환
df_AR = 외상매출금.toDF()
df_AR_Balance = 외상매출금잔액.toDF()

# 날짜형으로 변환
df_AR['전표일자'] = pd.to_datetime(df_AR['전표일자'])
결산일자 = dt.datetime(2018,12,31)
df_AR.insert(1, '결산일자', 결산일자)

# 거래처·전표일자 기준 정렬 ---
df_AR = df_AR.sort_values(['거래처', '전표일자'])

# df_AR과 df_AR_Balance의 조인
df_조인 = pd.merge(df_AR, df_AR_Balance, how='left', on = ['거래처'])

df_조인 = df_조인.fillna(0)

# groupby : df_조인을 거래처로
gb_1 = df_조인.groupby(['거래처'])

# groupby 테이블에 ['누적'] 칼럼 삽입
df_조인['차변_누적'] = gb_1['차변금액'].cumsum()
df_조인['대변_누적'] = gb_1['대변금액'].cumsum()

# groupby 테이블에 ['역누적'] 칼럼 삽입
gb_2 = df_조인.iloc[::-1].groupby(['거래처'])
df_조인['차변_역누적'] = gb_2['차변금액'].cumsum()
df_조인['대변_역누적'] = gb_2['대변금액'].cumsum()

# groupby 테이블에 ['잔액'] 칼럼 삽입
df_조인['잔액'] = df_조인['전기이월'] + df_조인['차변_누적'] - df_조인['대변_누적']
```

```

']

# groupby 테이블에 ['기말잔액'] 즉, 잔액의 마지막 값을 모든 레코드에 삽입
df_조인['기말잔액'] = gb_1['잔액'].transform('last')

#거래처·전표일자 기준 정렬
df = df_조인.sort_values(['거래처', '전표일자'])

# 거래처별로 역산하여 요약 생성
results = []

for cust, sub in df.groupby('거래처', as_index=False):
    # 그룹 내 마지막 잔액
    remaining = sub['기말잔액'].iloc[-1]

    # 기말잔액이 0인(=잔여채권이 없는) 경우 건너뛰다
    if remaining == 0:
        continue

    source_idx = None

    # 역순으로 돌면서 차변금액 차감
    for i in reversed(sub.index):
        remaining = remaining - sub.at[i, '차변금액']
        if remaining <= 0:
            source_idx = i
            break

    if source_idx is not None:
        src_date = sub.at[source_idx, '전표일자']
        settle_date = sub['결산일자'].iloc[-1]
        overdue_days = (settle_date - src_date).days

    else:
        # 완전 소진된 경우(=잔여채권 없음)
        src_date = pd.NaT
        overdue_days = 0

    results.append({
        '거래처': cust,
        '잔여채권원천일자': src_date.date() if pd.notna(src_date) else None,
        '연체일수': overdue_days
    })

# --- 4) 결과 요약 출력 ---
summary_df = pd.DataFrame(results)
print(summary_df)

```

	거래처	잔여채권원천일자	연체일수
0	00113	2018-09-28	94
1	00115	2018-09-28	94
2	00118	2018-09-28	94
3	00119	2018-09-28	94
4	00120	2018-09-28	94
5	00121	2018-09-28	94
6	00122	2018-09-28	94
7	00124	2018-04-30	245
8	00127	2018-09-28	94
9	00128	2018-07-31	153
10	00129	2018-09-28	94
11	00131	2018-09-28	94
12	00132	2018-04-30	245
13	00141	2018-06-29	185
14	00152	2018-09-28	94
15	00227	2018-09-28	94
16	00344	2018-09-28	94
17	00412	2018-09-28	94
18	00432	2018-09-28	94
19	00469	2018-08-31	122
20	00565	2018-09-28	94
21	00639	2018-09-28	94

13~15

당신은 (주)한공의 외부감사인이다. (주)한공은 ERP 프로그램을 사용하고 있으며 회계모듈에서 입수한 분개장을 입수하여 분석을 하려고 한다. 2010년 분개장을 csv로 받은 것이 분개장_08회_1급_1.csv 이며 이는 다음을 누르면 다운로드 받을 수 있다.

[분개장_08회_1급_1.csv](#)

분개장_08회_1급_1.csv 을 pandas의 데이터프레임으로 불러온다. Fraudit 테이 블로 바로 import를 하면 레코드가 10000개로 제한되므로 pandas를 이용하여야 한다. 데이터프레임명은 'df_전체'라고 한다. 데이터프레임으로 불러온 후 데이 터 타입을 살펴보면 다음과 같다.

칼럼명	데이터 타입	설명
전표번호	int64	전표번호
전표일자	int64	전표일자
증빙일자	int64	증빙일자요
전표라인번호	int64	동일한 전표번호내의 라인번호
차대변지시자	object	차대변지시자(S:차변, H:대변)
현지통화금액	float64	현지통화금액
통화키	object	KRW(원화)
계정코드	int64	계정코드
계정과목	object	계정과목
차변금액	float64	차변금액
대변금액	float64	대변금액
거래처코드	float64	거래처코드
사용자ID	object	담당자 ID

동일한 전표일자에 동일한 전표번호는 하나의 전표세트를 의미한다. 즉, 아래의 네모 선 안에 있는 것과 같이 동일한 전표일자와 동일한 전표번호를 가지는 세트를 하나의 전표세트라고 한다. 아래의 네모 선 안에 있는 하나의 전표세트의 레코드수는 3개이다.

	전표일자	전표번호	전표라인번호	생성부서	생성자	계정코드	계정과목	차변금액	대변금액
0	2024-01-01	2024000110		1 영업부	70036	4020 내수상환대출금		0	9164
1	2024-01-01	2024000110		2 영업부	70036	2119 부가세환수금		0	916
2	2024-01-01	2024000110		외영업부	70036	1270 내수외상환대출금		10000	0
3	2024-01-01	2024000119		1 기획부	120188	1215 보통예금		3800000	0
4	2024-01-01	2024000119		2 기획부	120188	1270 내수외상환대출금		0	3800000
5	2024-01-01	2024000216		1 영업부	60009	2087 거래보통금		17719396	0
6	2024-01-01	2024000216		2 영업부	60009	1270 내수외상환대출금		0	17719396

13 다음은 계정과목 중 외상매출금과 매출로 간주하는 항목만을 발췌한 표이다.

계정코드	계정과목	간주계정과목	계정코드	계정과목	간주계정과목
10080	외상매출금_1	외상매출금	10081	외상매출금_2	외상매출금
10082	외상매출금_3	외상매출금	10083	외상매출금_4	외상매출금
10084	외상매출금_5	외상매출금	10085	외상매출금_6	외상매출금
40001	제품매출액_1	매출	40002	제품매출액_2	매출
40003	제품매출액_3	매출	40004	상품매출액_1	매출
40005	상품매출액_2	매출	40006	상품매출액_3	매출
40007	반제품매출액_1	매출	40008	반제품매출액_2	매출
40009	부산물매출액_1	매출	40010	원재료매출액_1	매출
40011	부재료매출액_1	매출	40012	부재료매출액_2	매출
40013	용역매출액_1	매출			

차변 간주계정과목이 외상매출금이면서 차변금액이 0을 초과하거나 대변금액이 0미만이고, 대변 간주계정과목이 매출이면서 차변금액이 0 미만이거나 대변금액이 0을 초과하는 것이 포함된 전표세트의 개수를 숫자만 나타내라. 단, 천단위 콤마는 나타나지 않는다. (5점)

 정답 : 53496

해설

- (1) **New Script** 아이콘을 선택한다.
- (2) 아래와 같이 코드를 입력하고 해당 영역을 선택한 후  를 누른다.

```

# 판다스 라이브러리 불러오기
import pandas as pd

# 파일 импорт
df_전체 = pd.read_csv("분개장_08회_1급_1.csv", encoding = 'utf-8', sep = ',')
# 간주 현금 계정코드 리스트 처리
외상매출금_계정코드 = [10080, 10081,10082,10083,10084,10085]

# 간주 매출 계정코드 리스트 처리
매출_계정코드 = [40001, 40002, 40003, 40004, 40005, 40006, 40007, 40008, 40009,
                  40010, 40011, 40012, 40013]

# 외상매출금_차변빈도, 매출액_대변빈도 칼럼 삽입
df_전체['외상매출금_차변'] = [1 if i[0] in 외상매출금_계정코드 and (i[1] > 0 or i[2] <
0) else 0 for i in zip(df_전체['계정코드'], df_전체['차변금액'], df_전체['대변금액'])]

df_전체['매출_대변'] = [1 if i[0] in 매출_계정코드 and (i[1] < 0 or i[2] > 0) else 0
                        for i in zip(df_전체['계정코드'], df_전체['차변금액'], df_전체['대변금액'])]

# groupby : 전표번호, 전표일자
gb_전표번호_일자 = df_전체.groupby(['전표일자', '전표번호'])

# 전표_인덱스 칼럼 삽입
df_전체['전표_인덱스'] = gb_전표번호_일자.ngroup()

# groupby : 전표번호, 전표일자
gb_전표번호_일자 = df_전체.groupby(['전표_인덱스'])

# df_외상매출금매출_총괄 dataframe 만들기
df_외상매출금매출_총괄 = pd.DataFrame(
    {'외상매출금_차변_빈도': gb_전표번호_일자['외상매출금_차변'].sum(),
     '매출_대변_빈도': gb_전표번호_일자['매출_대변'].sum(),
     '차변합계': gb_전표번호_일자['차변금액'].sum(),
     '대변합계': gb_전표번호_일자['대변금액'].sum()}
    ).reset_index()

# 외상매출금_차변_빈도가 0 초과하면서 매출_대변_빈도가 0 초과하는 경우 발체
df_외상매출금매출_발체 = df_외상매출금매출_총괄[(df_외상매출금매출_총괄['외상매출금_차
변_빈도'] > 0) & (df_외상매출금매출_총괄['매출_대변_빈도'] > 0)]

# 상세분석 리스트
상세분석_리스트 = [i for i in df_외상매출금매출_발체['전표_인덱스']]
print(len(상세분석_리스트))

```

53496

- 14 동일한 거래처에 대한 거래금액 중 가장 큰 금액과 두 번째로 큰 금액과의 비율을 검토하게 되면 가장 큰 금액이 어느 정도 예외적으로 큰 금액인지를 직관적으로 가늠할 수 있다. 예를 들어, 어떤 거래처에 대한 외상매입금의 두 번째 금액이 1,000,000원인데 가장 큰 금액이 100,000,000원이라고 가정하자. 통상적으로 거래처에 대한 거래금액의 급격한 변화는 매우 특이한 상황이라고 볼 수 있다. 이러한 테스트를 **Relative Size Factor** 테스트라고 한다. 거래처별 '외상매입금_1' 계정과목 대변의 [가장 큰 금액 ÷ 두 번째로 큰 금액] 비율이 2를 초과하는 거래처의 개수를 숫자로만 나타내라. 천단위 콤마는 나타내지 않는다. (5점)

정답 : 16

해설

- (1) **New Script** 아이콘을 선택한다.
(2) 아래와 같이 코드를 입력하고 해당 영역을 선택한 후 를 누른다.

```
# 판다스 라이브러리 불러오기
import pandas as pd

# 파일 импорт
df_전체 = pd.read_csv("분개장_08회_1급_1.csv", encoding = 'utf-8', sep = ',')

# 외상매입금_1 계정과목 발취
df_외상매입금_1 = df_전체[(df_전체['계정코드'] == 20001) & (df_전체['대변금액'] > 0)]

# groupby : 거래처
gb_1 = df_외상매입금_1.groupby('거래처코드')

# 칼럼 삽입
df_외상매입금_1['거래처_인덱스'] = gb_1.ngroup()
df_외상매입금_1['대변_Max'] = gb_1['대변금액'].transform(lambda x :
                                                         x.nlargest(1).max())
df_외상매입금_1['대변_2nd_Max'] = gb_1['대변금액'].transform(lambda x :
                                                            x.nlargest(2).min())
df_외상매입금_1['대변_RSf'] = [ i[0]/i[1] if i[1] != 0 else -1
                              for i in zip(df_외상매입금_1['대변_Max'], df_외상매입금_1['대변_2nd_Max'])]

# RSF 요약 테이블 작성
df_RSf = pd.DataFrame(
    {'대변_Max': gb_1['대변_Max'].max(),
     '대변_2nd_Max': gb_1['대변_2nd_Max'].max(),
     '대변_RSf': gb_1['대변_RSf'].max()
    }).reset_index()
df_RSf_2 = df_RSf[df_RSf['대변_RSf'] > 2].reset_index()
print(df_RSf_2)
```

	index	거래처코드	대변_Max	대변_2nd_Max	대변_RSF
0	11	1320001.0	1632400.0	686400.0	2.378205
1	15	1510001.0	30637200.0	7392000.0	4.144643
2	16	1660001.0	179520.0	59840.0	3.000000
3	32	2322003.0	167200.0	75240.0	2.222222
4	39	2520001.0	3080000.0	990000.0	3.111111
5	41	2720001.0	22000000.0	10780000.0	2.040816
6	45	3020001.0	2200000.0	880000.0	2.500000
7	49	3330001.0	8594388.0	3812556.0	2.254233
8	57	3660001.0	95874146.0	40857484.0	2.346550
9	95	6660001.0	28072000.0	9002400.0	3.118280
10	108	7450001.0	22440000.0	10560000.0	2.125000
11	114	7760001.0	7810.0	3160.0	2.471519
12	118	8140003.0	4498560.0	1874400.0	2.400000
13	121	8300001.0	3009600.0	1073600.0	2.803279
14	134	9460001.0	2807574.0	1020096.0	2.752264
15	135	9500001.0	1188000.0	330000.0	3.600000

- 15 당신은 분개패턴을 분석하고자 한다. 이를 위해 'df_전체' 데이터프레임에 '계정코드_1' 칼럼을 다음과 같이 삽입한다.

```
>>> df_전체['계정코드_1'] = [ i[2] if i[0] > 0 and i[1] == 0
...                          else(-i[2] if i[0] < 0 and i[1] == 0
...                          else(-i[2] if i[0] == 0 and i[1] > 0
...                          else( i[2] if i[0] == 0 and i[1] < 0
...                          else( i[2] if i[0] == 0 and i[1] == 0
...                          else 0)))]
...
... for i in zip(df_전체['차변금액'],
...              df_전체['대변금액'], df_전체['계정코드'])
... ]
```

분개패턴은 하나의 전표세트에서 중복을 제외한 계정코드_1의 합계와 중복을 제외한 계정코드_1의 표준편차의 조합으로 구분한다고 가정한다. 중복을 제외하는 이유는 하나의 전표세트에서 여러 거래처에 대하여 중복되는 계정코드_1이 나오는 경우가 많기 때문에 이러한 영향을 제외하기 위한 것이다. 분개패턴을 기준으로 발생빈도가 2(즉, 1년 동안 발생한 빈도가 2)인 전표세트의 갯수를 숫자만 나타내라. 단, 천단위 콤마는 나타내지 않는다. 컴퓨터의 성능에 따라 코드를 처리하는데 3~6분 정도 걸릴 수 있다. (5점)

 정답 : 642

해설

- (1) **New Script** 아이콘을 선택한다.
(2) 아래와 같이 코드를 입력하고 해당 영역을 선택한 후 를 누른다.

```
# 판다스 라이브러리 불러오기
import pandas as pd

# 파일 импорт
df_전체 = pd.read_csv("분개장_08회_1급_1.csv", encoding = 'utf-8', sep = ',')

# 계정코드_1 칼럼 삽입
df_전체['계정코드_1'] = [ i[2] if i[0] > 0 and i[1] == 0
                        else(-i[2] if i[0] < 0 and i[1] == 0
                        else(-i[2] if i[0] == 0 and i[1] > 0
                        else( i[2] if i[0] == 0 and i[1] < 0
                        else( i[2] if i[0] == 0 and i[1] == 0
                        else 0)))]
                        for i in zip(df_전체['차변금액'], df_전체['대변금액'], df_전체['계정코드'])
                        ]

# groupby : 전표번호, 전표일자
gb_전표번호_일자 = df_전체.groupby(['전표일자', '전표번호'])
```

```

# 칼럼 삽입
df_전체['계정코드_1_합계_중복제외'] = gb_전표번호_일자['계정코드_1'].transform(
    lambda x: x.unique().sum())
df_전체['계정코드_1_표준편차_중복제외'] = gb_전표번호_일자['계정코드_1'].transform(
    lambda x: x.unique().std())
df_전체['계정코드_1_개수_중복제외'] = gb_전표번호_일자['계정코드_1'].transform(
    'nunique')
df_전체['전표_인덱스'] = gb_전표번호_일자.ngroup()

# groupby : 계정코드_1_합계_중복제외, 계정코드_1_표준편차_중복제외
gb_계정코드_1_중복제외 = df_전체.groupby(['계정코드_1_합계_중복제외', '계정코드_1_표
준편차_중복제외'])

# 인덱스 칼럼 삽입
df_전체['계정코드_1_중복제외_index'] = gb_계정코드_1_중복제외.ngroup()

# df_분개패턴 summarize 생성
df_분개패턴_1 = pd.DataFrame({
    '계정코드_1_중복제외_레코드': gb_계정코드_1_중복제외['계정코드_1_개수_중복제외'].max(),
    '계정코드_1_전표세트_중복제외_빈도': gb_계정코드_1_중복제외['전표_인덱스'].nunique(),
    '차변합계': gb_계정코드_1_중복제외['차변금액'].sum(),
    '차변Max': gb_계정코드_1_중복제외['차변금액'].max()
}).reset_index()

# 인덱스 칼럼 삽입
df_분개패턴_1.insert(0, 'S2_index', range(0, len(df_분개패턴_1)))

# 필터링
df_분개패턴_1_발취 = df_분개패턴_1[df_분개패턴_1['계정코드_1_전표세트_중복제외_빈도']
== 2]

print(len(df_분개패턴_1_발취))

```

642

- 16** 당신은 주요 기업의 재무제표 지표를 분석하고자 한다. 이를 위하여 아래의 사이트에서 csv를 다운로드 받는다. 아래 사이트에서 오른쪽 아래 CSV Export 버튼을 누르면 VALUEsearch.csv가 다운로드 된다.

<http://fda.valuesearch.co.kr/>

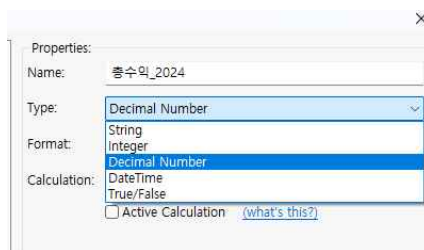
CSV 파일을 Fraudit 테이블로 변환한 후 아래와 같이 데이터타입을 변경한다.

칼럼명	데이터타입	설명
종목코드	String	종목코드
업체코드	String	업체코드
종목명	String	종목명(기업명)
유동자산_2023	Integer	2023년 유동자산 금액
유동자산_2024	Integer	2024년 유동자산 금액
비유동자산_2023	Integer	2023년 비유동자산 금액
비유동자산_2024	Integer	2024년 비유동자산 금액
자산총계_2023	Integer	2023년 자산총계
자산총계_2024	Integer	2024년 자산총계
유동부채_2023	Integer	2023년 유동부채 금액
유동부채_2024	Integer	2024년 유동부채 금액
비유동부채_2023	Integer	2023년 비유동부채 금액
비유동부채_2024	Integer	2024년 비유동부채 금액
부채총계_2023	Integer	2023년 부채총계
부채총계_2024	Integer	2024년 부채총계
자본총계_2023	Integer	2023년 자본총계
자본총계_2024	Integer	2024년 자본총계
총수익_2023	Integer	2023년 총수익
총수익_2024	Integer	2024년 총수익
매출원가_2023	Integer	2023년 매출원가
매출원가_2024	Integer	2024년 매출원가
판매비와관리비_2023	Integer	2023년 판매비와관리비
판매비와관리비_2024	Integer	2024년 판매비와관리비
법인세비용차감전계속영업이익_2023	Integer	2023년 법인세비용차감전계속영업이익
법인세비용차감전계속영업이익_2024	Integer	2024년 법인세비용차감전계속영업이익
계속영업이익_2023	Integer	2023년 계속영업이익
계속영업이익_2024	Integer	2024년 계속영업이익
당기순이익_2023	Integer	2023년 당기순이익
당기순이익_2024	Integer	2024년 당기순이익
영업활동으로인한현금흐름_2023	Integer	2023년 영업활동으로인한현금흐름
영업활동으로인한현금흐름_2024	Integer	2024년 영업활동으로인한현금흐름
종업원수_2023	Integer	2023년 종업원수
종업원수_2024	Integer	2024년 종업원수

매출총이익률은 (총수익 - 매출원가)/총수익으로 계산된다. 2024년 매출총이익률이 가장 높은 기업의 종목명을 입력하라. 매출총이익률은 소수점 넷째자리까지 구한다고 가정한다. (5점)

정답 : 엔씨소프트

해설

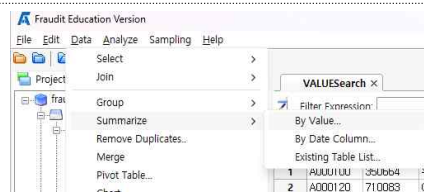


■ File >> Table Properties 메뉴를 누른다.



■ 총수익_2024 컬럼과 매출원가_2024 컬럼의 타입을 Decimal Number로 변경한다.

■ Data >> Summarize >> By value 메뉴를 누른다.



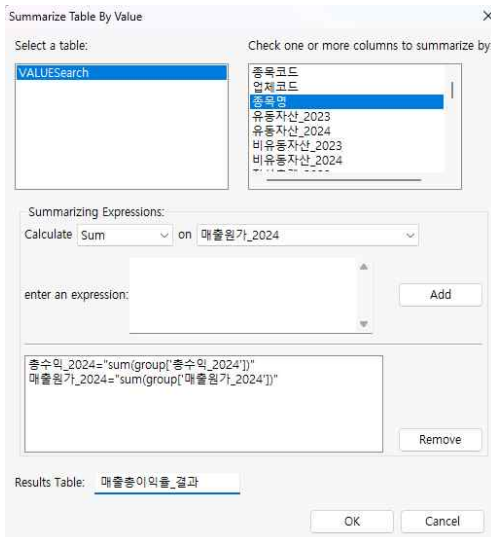
■ 종목명을 집계할 컬럼으로 선택

■ 집계 연산을 Sum(합계) 선택

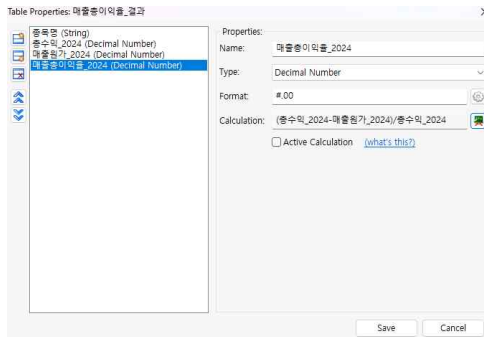
■ 총수익_2024, 매출원가_2024 컬럼을 각각 추가(Add)한 후 컬럼명 설정한다.

■ 결과 테이블명을 설정한 후 OK 버튼을 누른다.

■ 매출총이익율_결과 테이블 생성한다.



종목명	총수익_2024	매출원가_2024
0 CJ ENM	2787218126000	1496272293000
1 CJ대한물류	8366258767000	7395821660000
2 CJ제일제당	7598277073000	5554814532000
3 DB하이텍	591558677000	665948436000
4 DL	110207198000	7011650000
5 NCS	1100668248000	716005907000
6 E1	8999125094000	8622035371000
7 F&F	1517994384000	543612870000
8 F&F푸드스	69654061000	15556017000
9 GS리테일	11378032038000	8539950395000
10 HDC	71551849000	16270283000
11 HDC현대산업개발	4211972872000	3829431040000
12 HD한글조선해양	516796333000	425641407000



■ File >> Table Properties 메뉴를 누른다.



■ ‘매출총이익율_2024’ 컬럼 생성

- Calculation >> Expression Builder
- Expression: 입력

(총수익_2024 - 매출원가_2024)/총수익_2024

- OK 버튼을 누른다.

■ Save 버튼을 누른다.

종목명	총수익_2024	매출원가_2024	매출총이익율_2024	
0 엔씨소프트	1420139775000	3140401000	1.00	Sort ascending
1 NHN	3743929300000	17732895000	1.00	Sort descending
2 하림지주	904730000000	922000000	0.99	Table properties...
3 SK일리움	12774060000000	600190000000	0.95	
4 현대백화점	5245116069000	313170266000	0.94	
5 DL	110207198000	7011630000	0.94	
6 SKC	205662395000	13457937000	0.93	
7 크래프톤	2636975533000	338043825000	0.87	
8 현대홈쇼핑	2639086064000	363532449000	0.86	
9 신세계	2342996000000	335894686000	0.86	
10 HD현대	437639420000	68042525000	0.84	
11 KT	18579678000000	3328997000000	0.82	
12 LG유플러스	13496233000000	2711356000000	0.80	

■ 매출총이익율_2024 컬럼 기준으로 Sort descending 정렬한 결과 매출총이익률이 가장 높은 기업이 엔씨소프트인 것을 확인할 수 있다.

17 당신은 다음과 조건으로 이직 제의를 받았다.

- ① 제안받은 자리 : C그룹 지속가능경영본부 탄소배출 관리 매니저
- ② 연봉 보수 조건 :
 (대안 1) 기본급 연봉 320,000,000원 + 탄소크레딧 순이익의 3%(인센티브)
 (대안 2) 고정급 연봉 540,000,000원

당신은 다음과 같이 C그룹의 데이터 일부를 입수하였다.

- 탄소배출권 거래제에 참여하는 60개 산업 사이트 운영 중
- 분포 가정

구분	분포
탄소가격(CO ₂ t톤당)	Beta(2, 5, 20000, 45000)
사이트당 기준배출량	Tri(80000, 90000, 100000)
감축률	Beta(3, 4, 0.05, 0.30)
단위 감축비용	Tri(15_000, 18_000, 22_000)
고정비	Tri(1500000000, 2000000000, 2_200000000)

결과 변수의 산식은 다음과 같다.

구분	산식
사이트당 감축량	사이트당 기준배출량 * 감축률
총감축량	사이트당 감축량 * 사이트
탄소크레딧_수익	탄소가격 * 총감축량
총변동비	단위 감축비용 * 총감축량
총비용	총변동비 + 고정비
순이익	탄소크레딧_수익 - 총비용
인센티브	순이익 * 0.03

mcerp 라이브러리를 이용하여, 시뮬레이션을 10,000번 수행한 결과 (대안 1)이 540,000,000원 이상일 확률을 소수점 둘째 자리까지 반올림하여 구하라(예: 0.12345→0.12). 단, 시드는 np.random.seed(0)으로 고정시키고 다음과 같이 라이브러리를 불러온다. (6점)

```
import numpy as np
import pandas as pd
from pandas import *
import mcerp
from mcerp import *
```

```
import matplotlib.pyplot as plt
np.random.seed(0)
```

🔒 정답 : 0.29

해설

- (1) **New Script** 아이콘을 선택한다.
- (2) 아래와 같이 코드를 입력하고 해당 영역을 선택한 후 ▶를 누른다.

```
import numpy as np
import pandas as pd
from pandas import *
import mcerp
from mcerp import *
import matplotlib.pyplot as plt

# 시드 고정하기
np.random.seed(0) # seed를 0으로 고정시킨다.

# 입력변수의 파라미터 정하기
사이트 = 60
탄소가격 = Beta(2, 5, 20_000, 45_000)
사이트당_기준배출량 = Tri(80000, 90000, 100000)
감축률 = Beta(3, 4, 0.05, 0.30)
단위_감축비용 = Tri(15_000, 18_000, 22_000)
고정비 = Tri(1_500_000_000, 2_000_000_000, 2_200_000_000)

# 결과변수
사이트당_감축량 = 사이트당_기준배출량 * 감축률
총감축량 = 사이트당_감축량 * 사이트
탄소크레딧_수익 = 탄소가격 * 총감축량
총변동비 = 단위_감축비용 * 총감축량
총비용 = 총변동비 + 고정비
순이익 = 탄소크레딧_수익 - 총비용
인센티브 = 순이익 * 0.03
대안1 = 320_000_000 + 인센티브

# 대안1이 540000000원 이상일 확률 구하기
print(대안1 > 540_000_000)
```

0.29090000000000005

18 소매업체 A회사는 단일 제품을 판매하려 한다. 이 소매업체의 판매와 관련된 주요 절차와 내용은 다음과 같다.

- 주문수량 Q 를 결정한다. 주문수량은 0부터 1000개까지이다.
- 수요 D 는 불확실하며 $N(500, 100)$ 의 분포를 가지며 (-)수요는 없으므로 음수는 0으로 처리한다.
- 판매단가는 ₩1,000, 구매단가는 ₩400이다.
- 팔고 남은 재고인 잔여재고는 단위당 ₩500에 처분한다.
- 초과 수요(품질)는 단순히 기회손실로 취급하므로 벌과금은 없다.

A회사는 단위 주문수량 Q 에 대하여 다음의 기대이익을 최대화하는 최적의 주문수량 Q 를 구하라. 주문수량 Q 는 1개 단위로 변동된다고 가정한다.

$$\text{기대이익} = 1000 \times \min(Q, D) - 400 \times Q + 500 \times \max(Q-D, 0)$$

mcerp, geneticalgorithm, matplotlib 라이브러리를 이용하고, 시뮬레이션을 1,000번 수행하고, 시드는 np.random.seed(0)으로 고정시킨다. 따라서 다음과 같이 먼저 라이브러리를 불러오고 설정을 한다. (8점)

```
# 관련 라이브러리를 불러온다.
import numpy as np
import mcerp
from mcerp import *
from geneticalgorithm import geneticalgorithm as ga
# 시드 고정하기
np.random.seed(0)
# 시뮬레이션 횟수 정하기
mcerp.npts = 1000
```

 정답 : 416

해설

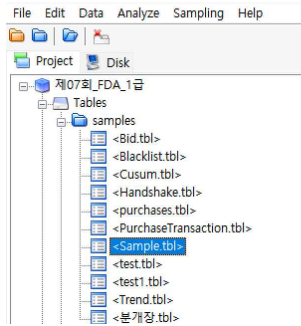
- (1) **New Script** 아이콘을 선택한다.
- (2) 아래와 같이 코드를 입력하고 해당 영역을 선택한 후 ▶를 누른다.

```
import numpy as np
import mcerp
from mcerp import *
from geneticalgorithm import geneticalgorithm as ga
# 시드 고정하기
np.random.seed(0) # seed를 0으로 고정시킨다.
# 시뮬레이션 횟수 정하기
mcerp.npts = 1000
# 입력변수의 분포 반영 및 array 변환
수요량 = Normal(500, 100)._mcpts
수요량_1 = np.where(수요량 < 0, 0, 수요량 )
# 목표함수 설정
def func(X) :
    # 전역변수 반영
    global 수요량_1
    # 결정변수_1
    주문수량 = X[0]
    # 결과변수 설정
    결과변수 = 1000*np.minimum(주문수량, 수요량) - \
                400*주문수량 + 500*np.maximum(수요량-주문수량, 0)
    return -np.mean(결과변수) # 최대화이므로 -를 반영한다.
varbound=np.array([[0,1000]])
model = ga(function = func,
            dimension = 1,
            variable_type = 'int', # 변동단위가 1%이므로 정수처리를 한다.
            variable_boundaries = varbound)
model.run()
```

The best solution found:
[416.]

Objective function:
-286024.2655939299

- 19 당신은 ㈜한공의 세전매출의 적정성을 테스트하기 위하여 MUS 샘플링을 수행하려고 한다. 관련 자료는 Sample.tbl이다. Sample.tbl은 Fraudit 교육용 버전을 설치하면 Project Folder >> samples folder가 생기고 그 안에 있다.



MUS 샘플링을 위한 파라미터 설정은 다음과 같다.

구분	설명
모집단의 화폐금액(Population)	‘세전매출’ 칼럼의 합계액
허용왜곡표시(Tolerable Error)	3%
예상오차율(Expected Error)	0.3%
Confidence Level	95%

Random starting point는 10000을 입력하고 샘플을 추출한다.

한편, 샘플에 포함된 다음의 송장번호의 세전매출 금액과 입증감사후 세전매출 금액의 내용은 다음과 같다.

송장번호	영업사원번호	CUSTNO	제품코드	세전매출	세전매출_Audit
A900033	101	21339	3	15,815.48	10,000
A900059	102	21256	4	188,381.16	150,000

Fraudit을 이용하여 샘플 추출한 후 이를 분석한 결과가 다음과 같았다.

Reject : the population as materially misstated.

population amount(5,307,370.36) × Tolerable error(3.0%) = 159,221.0

UpperErrorLimit : ㉠

159,221.0 < ㉡

빈칸 ㉠에 들어갈 금액을 입력하라. 단, Fraudit에서 나타내는 방식으로 천단위 콤마와 소수점 2째자리까지 입력하라. (6점)

정답 : 195,595.16

- 20 (주)한공의 분개장에서 상품매출이 발생한 레코드만 발체한 것이 FDA_08회_1급_상품매출.tbl이다. 이는 아래에서 받을 수 있다.

[FDA_08회_1급_상품매출.tbl](#)

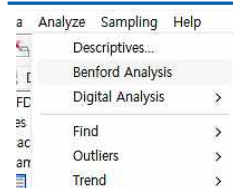
상기 파일을 불러온 후의 테이블 구조는 다음과 같다.

칼럼명	데이터 타입	설명
전표번호	Integer	전표번호
전표일자	Integer	전표일자
계정코드	Integer	계정코드
계정과목	String	계정과목
차변금액	Integer	차변금액
대변금액	Integer	대변금액
사용자ID	String	사용자ID

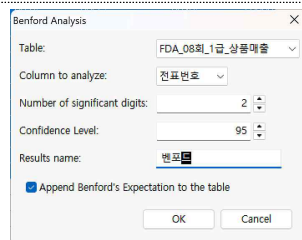
대변금액의 앞의 2단위(예: 대변금액이 12345이면 12를 의미함)에 대해서 벤포드 분석을 하려고 한다. 앞의 2단위의 벤포드 평균이 Expected라고 가정할 경우 실제 발견된 비율(Found Distribution)의 Z값이 5.0을 초과하는 항목의 개수는 몇 개 인지 숫자만 입력하라? 예를 들어 앞의 2단위가 10, 11, 12, 13이며 이들의 Z값이 5.0을 초과한다면 4를 입력한다. (6점)

정답 : 17

해설



(1) Analyze >> Benford Analysis 메뉴를 누른다.



- (2) Table : FDA_08회_1급_제품매출 선택
- (3) Column to analyze : 대변금액 선택
- (4) Number of significant digits : 2 선택
- (5) Confidence Level : 95% 선택(반드시 95%를 선택할 필요는 없다)
- (6) Results name : 벤포드 입력
- (7) Append Benford's Expectation to the table : 체크
- (8) OK 버튼을 누른다.

	First_2_Dig	Counts	Found	Expected	Z_score
0	54	399.0	.1106	.0080	69.2325
1	48	365.0	.1012	.0090	58.7055
2	33	294.0	.0815	.0130	36.3097
3	29	310.0	.0859	.0147	35.4378
4	57	193.0	.0535	.0076	31.7751
5	58	69.0	.0191	.0074	8.0900
6	17	20.0	.0055	.0248	7.3899
7	15	33.0	.0091	.0280	6.8212
8	11	66.0	.0183	.0378	6.0977
9	18	30.0	.0083	.0235	5.9611
10	37	4.0	.0011	.0116	5.8019
11	27	14.0	.0039	.0158	5.6731
12	87	42.0	.0116	.0050	5.5888
13	35	7.0	.0019	.0122	5.5491
14	42	5.0	.0014	.0102	5.1930
15	41	6.0	.0017	.0105	5.1139
16	34	11.0	.0030	.0126	5.0652
17	39	8.0	.0022	.0110	4.9764

(9) Z_score가 5.0을 초과하는 2단위 항목은 총 17개이다.

- 21 당신은 `random_text` 칼럼에 포함된 데이터를 분석하여 농축수산물의 항목별 금액 합계를 구하고자 한다. `random_text` 칼럼에 포함된 데이터만 추출하여 `csv` 파일로 변환한 것이 『`random_data.csv`』 파일이며 이는 아래의 링크를 누르면 다운로드 받을 수 있다.

[random_data.csv](#)

『`random_data.csv`』의 레코드 수는 3000개이며 데이터의 1번째 ~ 4번째 레코드는 다음과 같다.

054-458-9726	여	78,730,555원	김서연	대전광역시
140,520,362원	062-984-9500	경기도	최정웅	여
여	대전광역시	박정남	062-802-6086	110,101,416원
여	대전광역시	186,672,309원	오진우	041-758-1934

광역주소는 {'서울특별시', '부산광역시', '대구광역시', '인천광역시', '광주광역시', '대전광역시', '울산광역시', '세종특별자치시', '경기도', '강원도', '충청북도', '충청남도', '전라북도', '전라남도', '경상북도', '경상남도', '제주특별자치도'} 중 하나로 구성되어 있다.

레코드에 입력된 내용은 전화번호, 이름, 성별, 광역주소, 연봉이 랜덤으로 섞여 들어가 있다. 『`random_data.csv`』를 데이터프레임으로 불러온 후 광역주소별 연봉합계 최고지역을 입력하라. 예를 들어 최고지역이 서울특별시이면 서울특별시라고 입력하라. (8점)

 정답 : 전라북도

해설

- (1) **New Script** 아이콘을 선택한다.
- (2) 아래와 같이 코드를 입력하고 해당 영역을 선택한 후 를 누른다.

```
import pandas as pd
# 1) CSV 읽기 (헤더 없음, 모든 값을 문자열로)
df = pd.read_csv('random_data.csv', header=None, dtype=str)
```

```

# 2) 지역 리스트 정의 (발생 가능한 모든 지역)
regions = {
    '서울특별시', '부산광역시', '대구광역시', '인천광역시', '광주광역시',
    '대전광역시', '울산광역시', '세종특별자치시', '경기도', '강원도',
    '충청북도', '충청남도', '전라북도', '전라남도', '경상북도',
    '경상남도', '제주특별자치도'
}

# 3) 데이터 'melt' 각 행의 모든 셀(value) 정보를 하나의 테이블로
melted = (df.reset_index()                # 원래 행 번호를 'index' 컬럼으로 보존
          .melt(id_vars='index',          # index(행 번호)별로
                value_name='val')        # 모든 값을 'val' 컬럼에 모음
          .dropna(subset=['val']))        # 혹시 NaN이 있으면 제거
)

# 4) 지역 추출
region_series = (
    melted[melted['val'].isin(regions)]
    .drop_duplicates('index')           # 한 행당 하나만
    .set_index('index')['val']          # Series: index → 지역
)

# 5) 연봉 추출 (정규식 매칭 후 숫자만 남겨 int 변환)
pattern = r'^[₩]?(?:(?:\d{1,3}(?:,\d{3})+)|\d+)(?:원)?$'
salary_series = (
    melted[melted['val'].str.match(pattern, na=False)]
    .set_index('index')['val']
    .str.replace(r'[₩,원]', '', regex=True)
    .astype(int)
)

# 6) 행 번호(index) 기준으로 지역·연봉 병합
parsed = pd.DataFrame({
    'region': region_series,
    'salary': salary_series
}).dropna(subset=['region', 'salary'])

# 7) 지역별 통계 집계
agg = parsed.groupby('region')['salary'].agg(['sum', 'mean', 'count'])

# 8) 최고 합계·최고 평균 지역 찾기
max_sum_region = agg['sum'].idxmax()

print(max_sum_region)

```

전라북도

22 다음은 옵션과 관련된 내용이다.

- 아시아 콜옵션
- 현재주가 : 1
- 행사가격 : 취득일부터 만기까지의 평균가격
- 무위험이자율 : 6%
- 배당률 : 0%
- 만기 : 3년
- 변동성 : 20%
- 스텝(노드) : 3
- $dt = \text{만기/스텝(노드)}$
- 시뮬레이션 횟수 : 10000

기하 브라운 운동에 따른 몬테카를로 시뮬레이션 방법을 이용하여 콜옵션 가치를 소수점 3째 자리에서 반올림하여 구하라(예 : 0.12345→0.123). 단, 소수점 처리를 위해서 코드 상단에 `np.set_printoptions(precision=4, suppress=True)`를 입력하고, 시뮬레이션의 seed는 1로 설정하라. (6점)

 정답 : 0.118

해설

```
# 필요한 라이브러리 불러오기
>>> import numpy as np
>>> import math as m
# 기초변수 설정
>>> 최초주가 = 1
>>> 무위험이자율 = 0.06
>>> 변동성 = 0.20
>>> 배당률 = 0
>>> 만기 = 3 # 년
>>> 노드 = 3
>>> dt = 만기/노드 # 적어도 노드는 만기보다는 같거나 커야 하며 만기의 배수 단위로 한다.
# 시뮬레이션 횟수 정하기
>>> N_SIMS = 10000
# 할인계수 산정하기
>>> 할인계수 = np.exp(-무위험이자율 * dt) # rf는 risk free(무위험 이자율)의 약자이다.
# 기하 브라운 모형(GBM)에 따라 변화하는 주가를 나타내는 사용자 함수
>>> def simulate_gbm(s_0, mu, sigma, n_sims, T, N):
...     dt = T/N
...     np.random.seed(seed=1)
...     dZ = np.random.normal(scale = np.sqrt(dt), size = (n_sims, N))
...     Z = np.cumsum(dZ, axis = 1)
...     time_step = np.linspace(dt, T, N)
...     time_steps = np.broadcast_to(time_step, (n_sims, N))
...     S_t = s_0 * np.exp((mu - 0.5 * sigma ** 2) * time_steps + sigma * Z)
...     S_t = np.insert(S_t, 0, 최초주가, axis = 1)
...     return S_t
```

```

# 상기 사용자 함수에 기초변수를 넣어 array를 생성
>>> gbm_sims = simulate_gbm(s_0 = 최초주가, mu = 무위험이자율, sigma = 변동성,
...                          n_sims = N_SIMS, T = 만기, N = 노드)
# 행사가격 구하기 : 아시안이므로 만기일까지의 평균주가를 구한다.
>>> 행사가격 = np.mean(gbm_sims, axis=1) # 1차원 어레이로 결과가 나온다.
>>> 행사가격 = np.array([행사가격]) # 행렬변환 전단계 처리
>>> 행사가격 = 행사가격.T # 브로드캐스트를 위한 행렬변환
# payoff matrix 생성하기
>>> payoff_matrix = np.round(np.maximum(gbm_sims - 행사가격 ,
...                                     np.zeros_like(gbm_sims)),2)
# 만기 시점의 칼럼의 payoff_matrix 값만 있는 매트릭스를 작성
>>> value_matrix = np.zeros_like(payoff_matrix)
>>> value_matrix[:, -1] = payoff_matrix[:, -1]
# 최종옵션가치 도출
>>> 전체할인계수 = np.exp(-무위험이자율*dt*노드) # 년 환산 할인해야 하므로 dt*노드를
변경
>>> option_premium = np.mean(value_matrix[:, -1] * 전체할인계수)
>>> print(option_premium)
0.11835778895697724

```